INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
BHUBANESWAR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FINAL PROJECT REPORT

QUICKFLOW

Autonomous Fulfillment Orchestration &
Intelligent E-Commerce Logistics Platform



SUBMITTED BY	SUBMITTED TO
Manavendra Gupta **B125161** **Branch: CSE-B**	**Sobhanayak Srichandan**

Full-stack engineering • Graph algorithms • OOP • Concurrency • Real-time systems

1. Abstract & Executive Summary

QuickFlow is an end-to-end e-commerce fulfillment and logistics platform that connects customer purchases with warehouse operations. It combines a customer storefront, an operations command center, a real-time backend gateway and a C++17 orchestration engine. The system demonstrates how web engineering, graph algorithms, task scheduling and concurrency control can coordinate order fulfillment.

Customers can browse products, place orders and follow delivery progress. Operations administrators can monitor orders, warehouse capacity, workers, automated guided vehicles (AGVs), task queues, fulfillment stages and simulated peak workloads. Node.js/Express provides API and WebSocket connectivity, React 19 and Tailwind CSS power the interfaces, and the C++ engine handles computationally intensive routing and scheduling logic.

The central idea is visibility and coordination: the same order lifecycle should be understandable to the customer and actionable by the operations team.

Project at a glance

Layer	Responsibility	Technology
Customer experience	Catalog, cart, checkout, estimates and tracking	React 19, Tailwind CSS
Operations console	KPIs, hubs, workers, inventory, scheduling and simulation	React 19, Tailwind CSS, Lucide
API & real-time gateway	REST endpoints and live event delivery	Node.js, Express, WebSocket
Orchestration engine	Routing, task scheduling and concurrent execution	C++17
Desktop packaging	Desktop shell for web interfaces, where configured	Neutralinojs

Accuracy note

This report uses the supplied project description, code excerpts and seven UI screenshots. Figures displayed in the UI—such as throughput, uptime and latency—are treated as demonstration or reported benchmark values unless supported by repeatable test logs. They should not be represented as production guarantees without evidence.

2. Problem Statement & Motivation

E-commerce fulfillment is more than a checkout operation. Once an order is placed, a system must select a fulfillment hub, reserve stock, create warehouse tasks, sequence work, monitor progress and communicate changes. These activities become difficult when multiple hubs, workers, AGVs, delivery deadlines and concurrent orders are involved.

Problems addressed

Problem	Impact	QuickFlow response
Facility selection	Poor sourcing decisions increase cost and delay.	Model hubs as a weighted graph and consider capacity indicators.
Queue congestion	FIFO can leave urgent orders waiting behind long tasks.	Offer FIFO, priority, EDF and hybrid policies.
Concurrent inventory access	Simultaneous reservations can cause inconsistent stock.	Use synchronized operations and controlled shared-state access.
Disconnected visibility	Customer and operations teams may see stale states.	Share an order lifecycle and broadcast updates over WebSockets.
Peak-load uncertainty	Teams cannot easily predict bottlenecks.	Provide what-if workload and scheduler simulations.

Motivation

The project brings frontend development, backend APIs, C++ systems programming, data structures, algorithms, object-oriented design and real-time communication into one practical workflow. It demonstrates these topics as cooperating parts of a logistics system rather

than isolated exercises.

Intended users

Customers need a clear purchase and delivery experience. **Operations administrators** need visibility into hubs, queues, staff, fleet activity and service levels. **Warehouse workers and AGVs** are modeled as execution resources whose availability affects fulfillment.

3. Objectives & Proposed Solution

Objective	Description
Order lifecycle visibility	Represent progress through guarded stages from creation to delivery.
Shortest-path routing	Use Dijkstra over a weighted hub network.
Flexible task scheduling	Adapt task order to urgency, deadlines and queue conditions.
Concurrent execution	Use thread-safe queues and synchronization primitives.
Live updates	Broadcast order and operational events through WebSockets.
Decision support	Display capacity, workload, worker availability and scenario results.
Unified experience	Connect storefront, tracking and operations console to one backend.

End-to-end workflow

Step	Action
1	Customer browses the storefront and submits an order.
2	Backend validates the request and creates the order.
3	Routing logic evaluates eligible hubs and selects a route.
4	Inventory is reserved and fulfillment tasks are generated.
5	Scheduler assigns tasks to available workers or AGVs.
6	Order state changes are reported to the backend gateway.
7	WebSocket events update customer tracking and admin views.
8	The order proceeds through quality check, shipment and delivery.

4. System Architecture & Technology Stack

Component	Responsibility
Customer Storefront	Catalog, product details, cart, checkout, delivery location and tracking.
Admin Command Center	Orders, fulfillment, tasks, warehouses, workers, inventory, scheduling, analytics and simulation.
Node.js / Express	API requests, validation and coordination with the engine.
WebSocket gateway	Live order-state and operational event delivery.
C++17 engine	Routing, task ordering, task execution and concurrency.
Domain/state layer	Consistent order, task, hub and worker models.

Technology rationale

C++17 supports explicit resource management, standard-library data structures and threading. **Node.js/Express** provides an event-driven API layer. **WebSocket** enables live updates without continuous polling. **React 19** supports reusable, state-driven UI components. **Tailwind CSS** supports consistent styling. **Neutralinojs** can package web interfaces as a desktop shell when configured to launch the required frontend/backend services.

Application modules

Application	Modules
Customer Storefront (port 3000)	Catalog, cart, checkout, delivery location and tracking.
Admin Console (port 3001)	Overview, orders, fulfillment, tasks, warehouses, workers, inventory, scheduling, simulation, analytics, events and settings.
Backend & C++ engine (port 5000 as described)	REST API, WebSocket subscriptions, routing, scheduling and concurrent execution.

Ports are taken from the supplied project overview and should be checked against the final runtime configuration.

5. End-to-End Workflow & Order Lifecycle

The order lifecycle is the backbone of QuickFlow. It connects customer actions to warehouse execution and provides a common sequence for the tracking interface and operations console.

Stage	Meaning	Typical action
1. CREATED	Order request recorded.	Create order and validate details.
2. CONFIRMED	Order passes confirmation checks.	Confirm customer/order details.
3. ALLOCATED	Hub and stock selected.	Reserve inventory and assign hub.
4. PICKING	Items collected from warehouse.	Execute picking tasks.
5. PACKING	Items consolidated and packed.	Prepare package.
6. QUALITY_CHECK	Package checked before dispatch.	Verify items and package integrity.
7. SHIPPED	Carrier handover completed.	Record dispatch and tracking.
8. DELIVERED	Order reaches destination.	Close fulfillment lifecycle.

State-machine guards

A strict state machine prevents invalid jumps. For example, an order should not move directly from CREATED to SHIPPED while allocation, picking, packing and quality checks remain incomplete. A transition guard validates whether the requested next state is allowed.

Customer journey

The customer selects a product, adds it to the cart, checks the delivery location and places an order. The tracking page displays the order identifier, current status, destination, courier details where available, a route visualization and milestone progress. The supplied screenshot shows an example order at PICKING, with earlier milestones completed and later milestones pending.

Operations journey

The operations console shows the same order with its assigned hub, SLA priority, current stage and remaining tasks. Multiple orders can be inspected at different lifecycle stages.

6. Warehouse Network & Dijkstra Routing

QuickFlow models fulfillment hubs as nodes in a weighted graph. Edges represent possible movement or transfer links. A route can be evaluated using a configured weight such as distance, transit time, tariff or a composite cost. The supplied topology view illustrates Delhi (NCR-01), Mumbai (BOM-04), Hyderabad (HYD-02) and Kolkata (CCU-03).

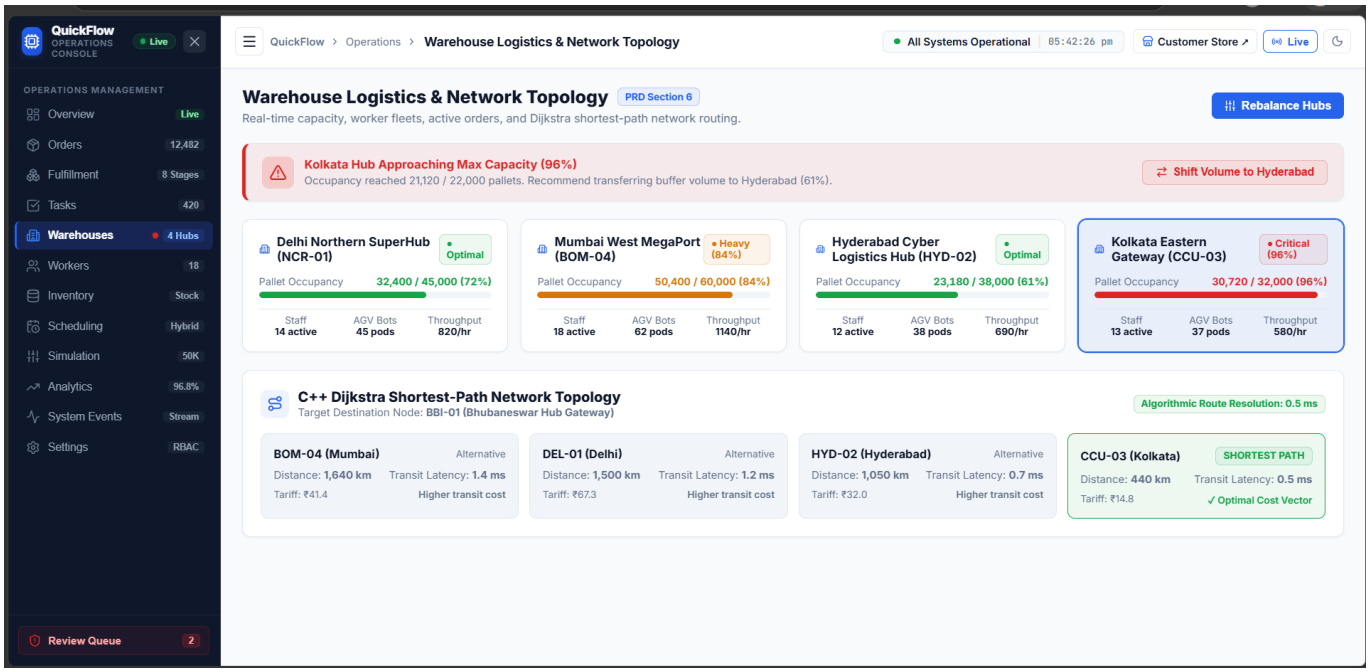


Figure 1. Warehouse capacity cards and shortest-path network topology from the application.

Dijkstra's algorithm

Dijkstra finds a minimum-total-weight path from a source node to reachable destinations when all edge weights are non-negative. It stores the best known distance to each node and uses a min-priority queue to process the closest candidate.

Code glimpse

```
using NodePair = std::pair; std::priority_queue, std::greater> pq; distances[startHub] = 0.0; pq.push({0.0, startHub});
while (!pq.empty()) { auto [d, u] = pq.top(); pq.pop(); if (d > distances[u]) continue; for (const auto& edge :
adjacencyList[u]) { double nd = d + edge.weight; if (nd < distances[edge.destinationHub]) {
distances[edge.destinationHub] = nd; previous[edge.destinationHub] = u; pq.push({nd, edge.destinationHub}); } } }
```

The predecessor map supports route reconstruction by walking backward from the destination. With an adjacency list and binary heap, complexity is O((V + E) log V), commonly simplified to O(E log V) for connected sparse graphs. Dijkstra is not valid for negative edge weights. The result is only as meaningful as the configured weight model: distance alone may not represent lowest cost or fastest delivery.

7. Fulfillment Pipeline & Task Scheduling

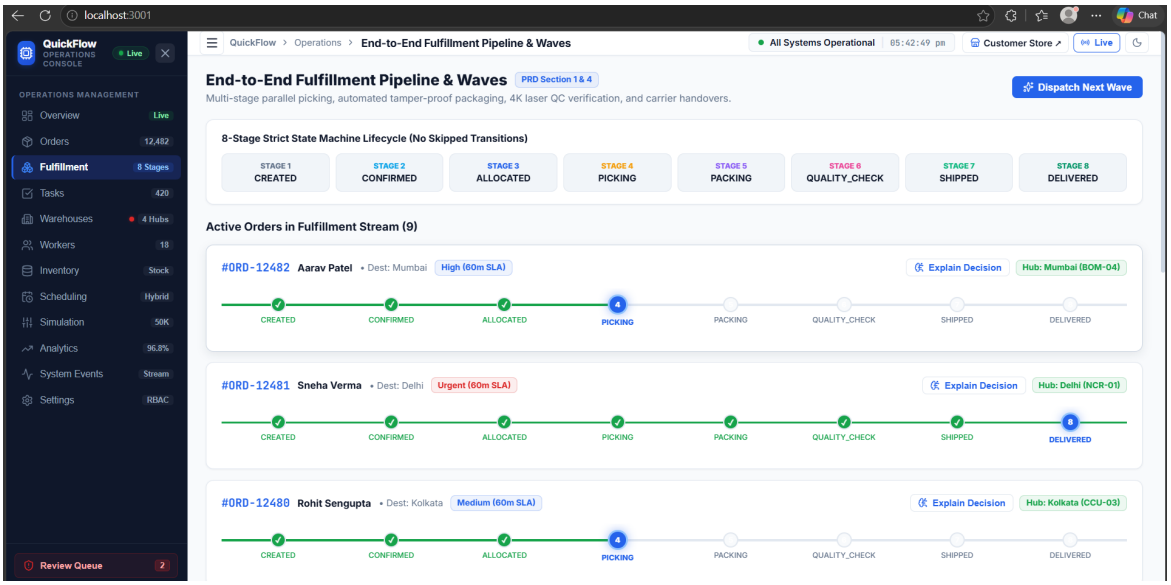


Figure 2. Eight-stage lifecycle and active order progress in the operations console.

The pipeline screen shows the eight stages and active orders at different progress points. Completed stages are marked, the current stage is highlighted and future stages remain pending. This helps operators identify stalled work and compare SLA priorities.

Scheduling strategies

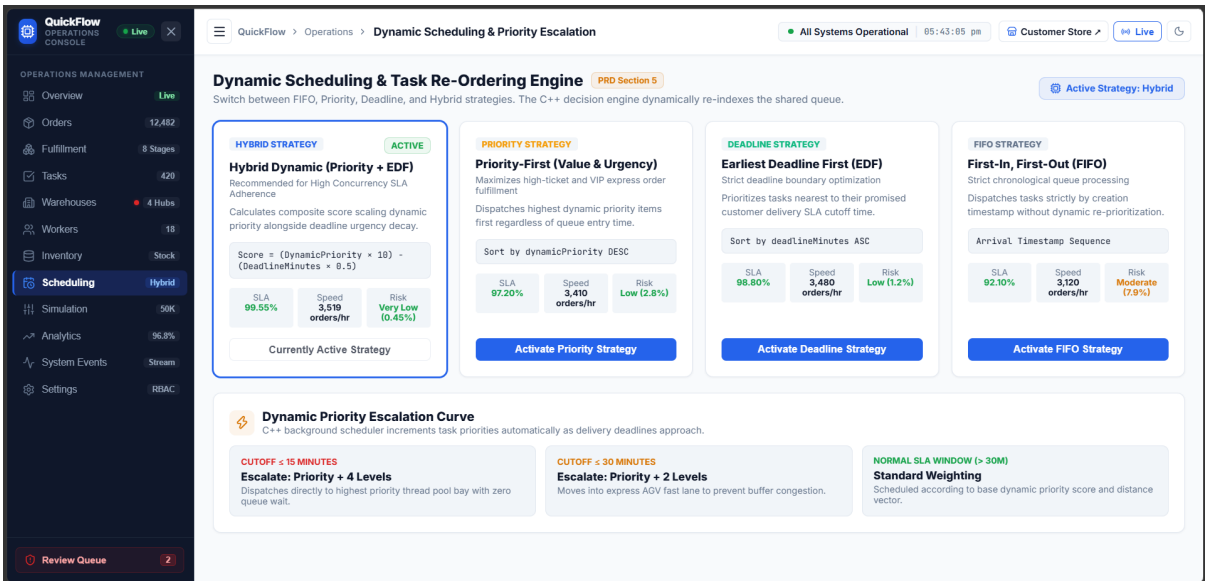


Figure 3. Dynamic scheduling strategy comparison and priority escalation.

Policy	Selection rule	Trade-off
FIFO	Oldest queued task first.	Simple; urgent work may wait.
Priority	Highest priority first.	Protects urgent work; starvation is possible.
EDF	Nearest deadline first.	Supports SLA adherence; depends on good deadline estimates.
Hybrid	Combines priority and deadline urgency.	Flexible; scoring weights need tuning.

Strategy-pattern code glimpse

```
auto it = std::max_element(taskQueue.begin(), taskQueue.end(), [](const auto& a, const auto& b) { return a->getPriorityLevel() < b->getPriorityLevel(); }); auto chosenTask = *it; taskQueue.erase(it); return chosenTask;
```

The Strategy pattern changes scheduling policy without rewriting task execution. Factory can create task objects, Observer/EventBus can publish events, and the State Machine can validate order transitions.

8. C++ Engine, OOP & Concurrency

The C++17 component is the computation-oriented part of QuickFlow. Based on the supplied description, it contains routing, pipeline scheduling and task-execution responsibilities. Clear interfaces between the engine and Node.js keep network concerns separate from algorithmic execution.

OOP concept	Application in QuickFlow
Encapsulation	Keep task state and scheduler internals behind class interfaces.
Inheritance & polymorphism	Represent picking, packing, QC and shipping tasks through a common Task abstraction.
Abstraction	Expose operations such as calculateShortestPath() without exposing internal structures.
Composition	Build orchestration from router, scheduler, queue and worker-pool components.
RAII	Tie lock/resource lifetime to object lifetime for reliable cleanup.

Concurrency and thread safety

A thread pool reuses a bounded number of worker threads. A thread-safe queue protects shared queue state with a mutex. A worker waits for work, removes one task while holding the appropriate lock, releases the lock and performs the task. Short critical sections reduce contention.

Synchronization code glimpse

```
{ std::unique_lock lock(queueMutex); condition.wait(lock, [&] { return stopping || !tasks.empty(); }); if (stopping && tasks.empty()) return; task = std::move(tasks.front()); tasks.pop(); } // Execute task outside the queue lock.
```

RAII locking releases the mutex when the lock object leaves scope. Correct synchronization also requires consistent lock ordering, safe shutdown, condition-variable predicates and careful shared-inventory handling. A mutex alone does not prove absence of deadlocks or race conditions; testing and review are required.

DAG execution

A Directed Acyclic Graph represents task dependencies. Packing waits for picking, and dispatch staging waits for quality approval. Independent picking tasks may run in parallel; dependent tasks become eligible only after prerequisites complete.

9. Backend Gateway & Real-Time Communication

The Node.js/Express gateway connects both React applications to the orchestration engine. REST endpoints suit request/response operations such as product lookup, order creation, inventory queries and administrative actions. The gateway can validate payloads, enforce authorization and translate engine results into API responses.

WebSocket event broadcasting

WebSockets maintain a persistent, bidirectional connection. Rather than repeatedly polling the server, connected clients can receive an event when an order changes. This is useful for customer tracking and the live operations feed.

Supplied code glimpse

```
export function broadcastOrderUpdate(order) { const payload = JSON.stringify({ type: 'ORDER_STATE_UPDATE', timestamp: new Date().toISOString(), order: { id: order.id, status: order.status, assignedHub: order.assignedHub, deliveryEta: order.deliveryEta, tasksRemaining: order.tasks ? order.tasks.filter(t => t.status !== 'DONE').length : 0 } }); wss.clients.forEach(client => { if (client.readyState === WebSocket.OPEN) client.send(payload); }); }
```

The server serializes a compact event and sends it to open client sockets. A production implementation should consider authentication, per-order authorization, reconnect handling, heartbeat/ping, event ordering, backpressure, validation and horizontal scaling through shared pub/sub.

Consistency and security

The backend/domain service should own authoritative order state. Admin and customer clients render updates from that source. If an event is missed, a client should fetch the latest snapshot through REST. Important controls include authenticated sessions, role-based access control, input validation, rate limiting, secure transport, secret management and audit logs. Delivery confirmation PINs should not appear in public logs.

10. Project Screenshots & Feature Demonstrations

Operations overview

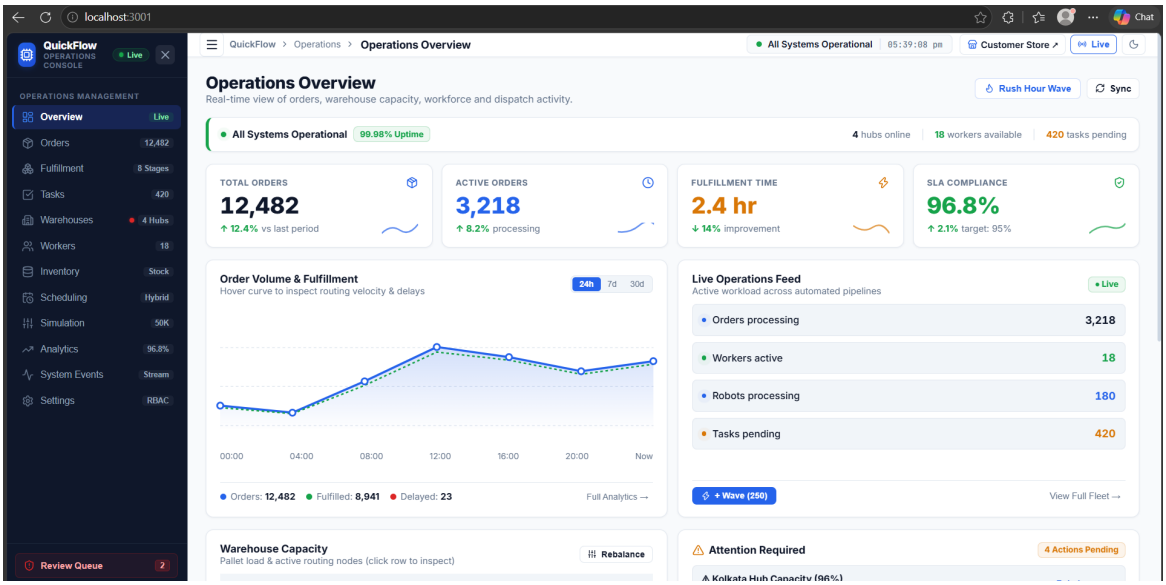


Figure 4. Admin overview: KPIs, order volume, live operations feed and attention items.

Customer order tracking

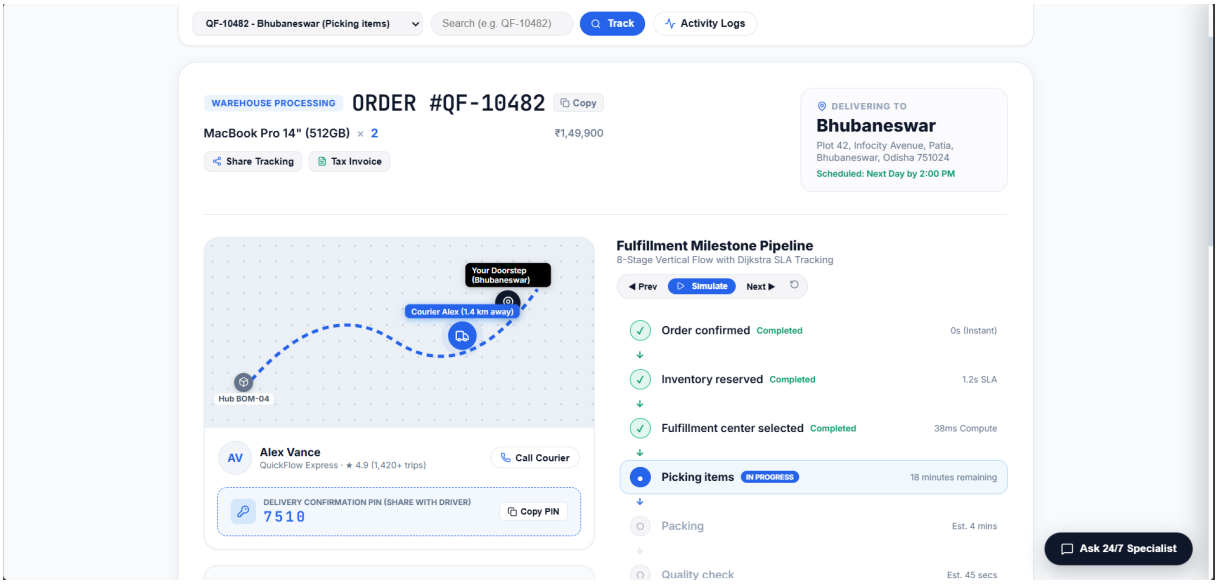


Figure 5. Customer tracking: order details, route visualization, courier information and milestones.

Customer storefront

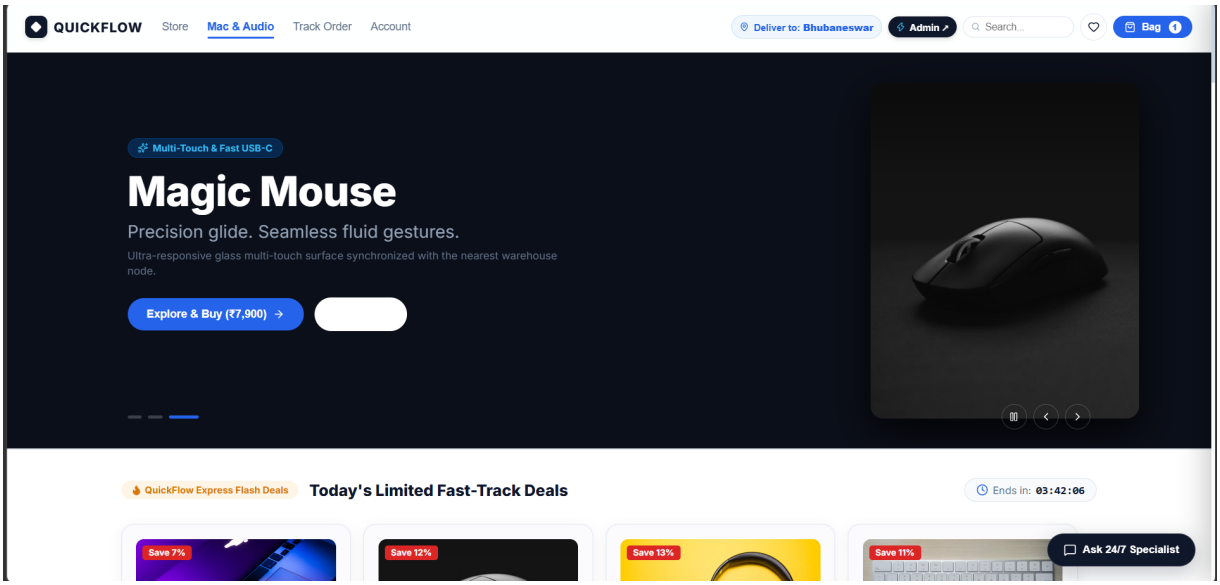


Figure 6. Storefront: category navigation, delivery location, product promotion and shopping actions.

11. Simulation, Results & Conclusion

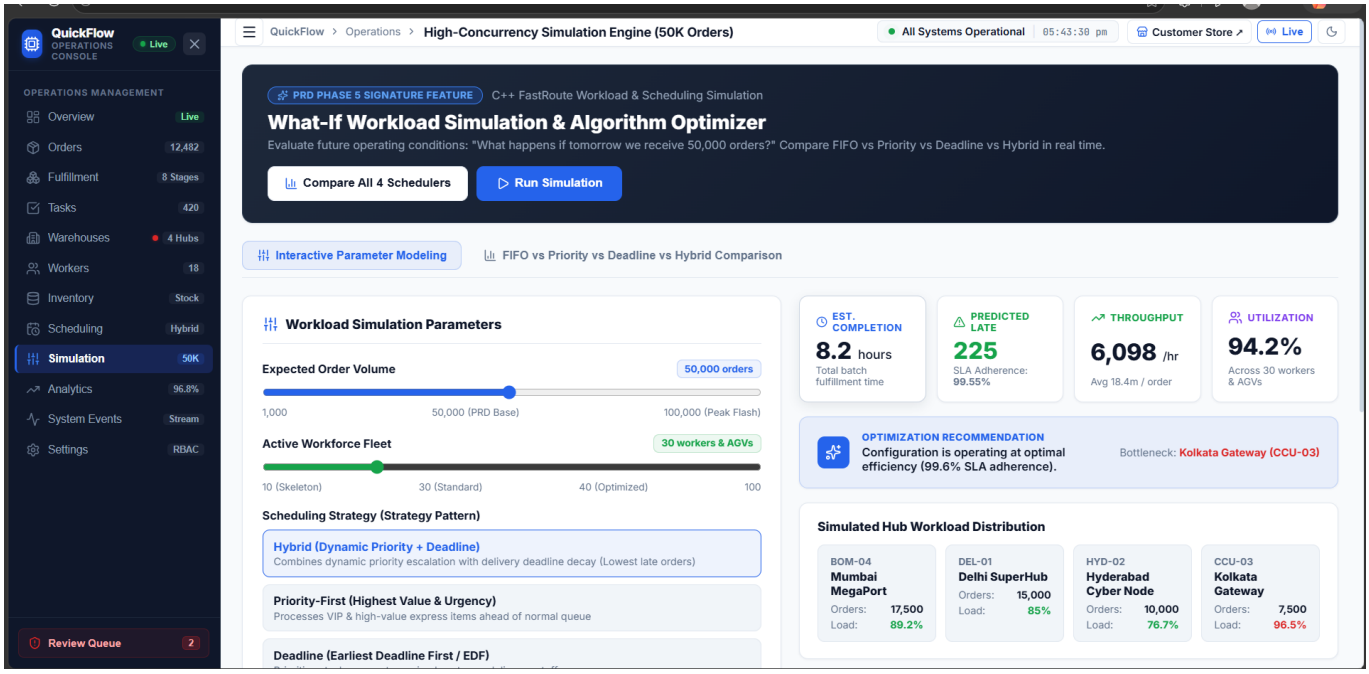


Figure 7. What-if workload simulation and scheduler comparison interface.

What-if simulation

The simulation interface allows an operator to vary expected order volume, workforce size and scheduling strategy, then inspect projected completion time, late orders, throughput, utilization and hub distribution. It supports comparative experiments, such as observing how a larger workforce or a different scheduling policy changes projected outcomes.

Reported demonstration metrics

Metric	Value described/shown	How to interpret
Dijkstra resolution	0.5–0.85 ms	Reported benchmark; validate with repeatable tests.
Simulation workload	50,000 orders	Scenario input, not proof of production throughput.
WebSocket latency	Under 45 ms	Reported result/target; document measurement method.

Metric	Value described/shown	How to interpret
Scheduler workload	50,000+ simulated subtasks	Reported capability; substantiate with logs.
SLA / uptime UI values	96.8% / 99.98%	Demo values; define calculation window and source.

Testing and future work

Recommended tests include state-transition unit tests, route reconstruction and unreachable-node tests, scheduler ordering and starvation tests, concurrency stress tests, WebSocket reconnect/integration tests and load tests with documented hardware, compiler flags, dataset and repeated runs. Future improvements include persistent transactional inventory, authentication, message-broker event distribution, observability, realistic routing data, retries/dead-letter handling, CI/CD and container orchestration.

Conclusion

QuickFlow demonstrates an integrated approach to e-commerce fulfillment: customers receive a clear order journey while operators gain visibility into hubs, workforce, tasks, routing and workload scenarios. Its technical value lies in combining modern web interfaces with C++ algorithms and concurrency concepts, connected through a real-time Node.js gateway.

Submission accuracy note

This report is based on the supplied project summary, code excerpts and screenshots. Verify each implementation detail against the final source tree. Replace demonstration metrics with measured results where possible, and add database schema, API examples, test evidence and deployment details if available.